

LLM Observability & Monitoring

Seeing inside your AI system in production — LLM tracing, prompt tracing, token/cost/latency tracking, error tracking, tool call and retrieval tracing, agent tracing, production monitoring, and debugging LLM applications.

DIFFICULTY LEVEL

Intermediate

ESTIMATED PRACTICE TIME

90-120 minutes

ESTIMATED READING TIME

55-70 minutes

PREREQUISITES

Modules 01-16

LEARNING OUTCOMES

By the end of this module, you will be able to: trace a request's full lifecycle through an LLM application; track token, cost, and latency metrics in production; monitor tool calls, retrieval steps, and agent behavior specifically; set up production monitoring with meaningful alerts; and debug a failing LLM application systematically instead of guessing.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	LLM Tracing
3.2	Prompt Tracing
3.3	Token, Cost & Latency Tracking
3.4	Error Tracking
3.5	Tool Call Tracing
3.6	Retrieval Tracing
3.7	Agent Tracing
3.8	Production Monitoring
3.9	Debugging LLM Applications
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

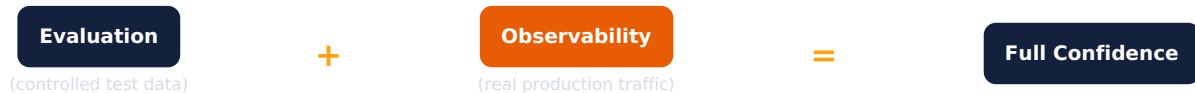
1. Introduction

What is this topic?

Module 16 taught you how to evaluate whether your AI system is good, using controlled test sets. This module is about seeing what's actually happening in production, live — every real request, every tool call, every retrieval, every cost — the visibility layer that evaluation alone can't provide.

Why is it important?

Evaluation runs on datasets you chose. Observability watches everything real users actually send — including the inputs you never thought to test for. Without it, production problems are invisible until a user complains, and even then, you're often debugging blind.



Why should AI Engineers learn it?

Complex AI systems (RAG, Module 07; agents, Module 08) have many moving parts, any of which can fail independently. Observability is what lets you pinpoint exactly which part failed on which real request, instead of re-running an entire multi-step pipeline blind, guessing.

Where is it used in the real world?

Every production AI system at meaningful scale needs tracing and monitoring — it's how teams catch cost spikes, latency regressions, and silent quality drops before they become major incidents.

Why is it relevant today?

As LLM applications have grown more complex (multi-step agents, RAG pipelines, tool chains), dedicated LLM observability tooling has become a standard, expected part of the AI engineering stack — not an afterthought bolted on later.

Note

This module pairs directly with Module 16 — evaluation tells you if your system is good on data you chose; observability tells you what's actually happening on data you didn't choose (real users).

2. Before You Begin

RECAP

Tokens, cost, latency (Module 01, 3.4, 3.10; Module 09, 3.4-3.6) — this module tracks these same quantities continuously, in production, rather than as one-off estimates.

RECAP

Agent loops and tool use (Modules 06, 08) — tracing these multi-step systems is one of this module's core focuses, since they have the most places to go wrong.



Tip

Add basic tracing to a project from day one, even a simple print-to-log version — retrofitting observability onto a system already in production is much harder than building it in from the start.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

🔍 3.1 LLM Tracing

DEFINITION

LLM tracing is recording the complete path a single request takes through an AI system — every prompt, model call, tool call, and intermediate step, in order, tied together as one traceable unit.

SIMPLE EXPLANATION

A "trace" is the full story of one request from start to finish. For a simple chatbot, that might be one prompt and one response; for an agent (Module 08), it could be a dozen steps — tracing captures all of them under one identifiable request ID.

💡 *Like a package's tracking number that shows every checkpoint it passed through — origin, sorting facility, transit hub, delivery — rather than just "shipped" and "delivered" with nothing in between.*

EVERYDAY EXAMPLE

A trace for one user question to a RAG chatbot might show: query received → embedded → 3 chunks retrieved → prompt assembled → model called → response returned, all under one trace ID.

WHY IT MATTERS

Without tracing, you only see the final input and output — every intermediate step (Module 07's retrieval, Module 06's tool calls) is invisible, making it nearly impossible to pinpoint where a multi-step failure actually happened.

COMMON MISTAKES

Only logging the final response, discarding the intermediate steps that would have explained why a bad response happened.

BEST PRACTICES

Give every request a unique trace ID and log every intermediate step against it, from day one of a project.

SUMMARY

LLM tracing captures the full step-by-step story of a single request — the foundational data every other concept in this module builds on.

📄 3.2 Prompt Tracing

DEFINITION

Prompt tracing is specifically recording the exact prompt (system + user messages, with any injected context, Module 05) sent to the model for every single request.

SIMPLE EXPLANATION

Because prompts are often assembled dynamically (templates, Module 02, 3.9; retrieved context, Module 07), the actual prompt sent can differ meaningfully from what you'd expect just by reading your code — prompt tracing captures the real, final version.

💡 *Like recording the exact final draft of a letter that was actually mailed, not just the template it started from — especially important when the letter gets auto-filled with different details each time.*

EVERYDAY EXAMPLE

Discovering, via prompt tracing, that a retrieval bug (Module 07) was silently injecting an

empty context block into 5% of prompts — invisible without seeing the actual sent prompt.

WHY IT MATTERS

When a response looks wrong, the first debugging question is almost always "what did the model actually receive?" — prompt tracing answers that directly instead of requiring you to guess or reconstruct it.

COMMON MISTAKES

Only logging the template or code, not the fully-assembled, dynamically-injected final prompt actually sent.

BEST PRACTICES

Log the complete, final prompt (with sensitive data redacted as needed) for every request, not just the static template.

SUMMARY

Prompt tracing captures the actual, final prompt sent — essential for debugging dynamic, template-based, or context-injected systems.

💰🔪 3.3 Token, Cost & Latency Tracking

DEFINITION

Token, cost, and latency tracking is continuously measuring these three quantities per request in production, rather than only estimating them ahead of time (Module 09, 3.4-3.6).

SIMPLE EXPLANATION

Estimates from Module 09 tell you what to expect; production tracking tells you what's actually happening — real traffic patterns, real edge cases, and real drift over time that estimates can't fully predict in advance.

```
Request #4821: tokens_in=340  tokens_out=120  cost=$0.0019  latency=1.4s
Request #4822: tokens_in=8200 tokens_out=95   cost=$0.0246  latency=3.8s  ▲ outlier
```



Like the difference between a car's estimated gas mileage on the sticker versus the actual mileage you track on every real drive — real conditions reveal patterns the estimate never could.

EVERYDAY EXAMPLE

Discovering via production tracking that 2% of requests have unusually high token counts (users pasting huge documents), driving a disproportionate share of total cost.

WHY IT MATTERS

Cost and latency estimates are approximations; only production tracking reveals real distributions, outliers, and trends over time — the data you actually need for informed optimization decisions (Module 09).

COMMON MISTAKES

Only tracking averages, missing outliers that may represent a disproportionate share of total cost or a specific bad user experience.

BEST PRACTICES

Track distributions (not just averages) — p50, p95, p99 latency and cost are far more informative than a single mean value.

SUMMARY

Continuous production tracking of tokens, cost, and latency reveals real-world patterns and outliers that pre-launch estimates can never fully capture.

🧩 3.4 Error Tracking

DEFINITION

Error tracking is systematically recording every failure in an AI system — API errors, timeouts, validation failures (Module 04, 10), tool failures (Module 06) — with enough context to diagnose each one.

SIMPLE EXPLANATION

Not every failure is a crash a user notices immediately; many are silent (a validation check quietly rejecting output, a tool call timing out and falling back) — error tracking makes all of these visible and countable.

 *Like a factory's defect log that records every quality-control rejection, not just the ones a customer happened to notice and complain about.*

EVERYDAY EXAMPLE

Tracking that output validation (Module 04, 10) rejected 3% of responses this week, up from 1% last week — a signal worth investigating even though no user directly complained.

WHY IT MATTERS

Silent failures (a fallback kicking in, a retry succeeding on the second try) can mask a growing underlying problem if they're not tracked and trended over time.

COMMON MISTAKES

Only tracking hard crashes, missing soft failures (validation rejections, silent fallbacks) that are equally worth monitoring.

BEST PRACTICES

Track and categorize every failure type, including recovered ones (successful retries, fallbacks); trend error rates over time to catch gradual degradation.

SUMMARY

Error tracking makes every failure — visible or silent — countable and trendable, catching problems before they become major incidents.

✂3.5 Tool Call Tracing

DEFINITION

Tool call tracing is recording every tool invocation (Module 06) an LLM makes — which tool, what arguments, what result, and how long it took.

SIMPLE EXPLANATION

Since tool calls involve real external systems (APIs, databases), they're often where failures actually occur even when the LLM's reasoning was fine — tool call tracing isolates this specific layer for debugging.

 *Like logging every phone call a customer service rep makes to a supplier during a support case — separate from the conversation with the customer, this record shows exactly which external interaction may have caused a delay or error.*

EVERYDAY EXAMPLE

Discovering via tool call tracing that a weather API tool (Module 06, 3.3) was timing out on 8% of calls, explaining a cluster of degraded agent responses.

WHY IT MATTERS

Tool call tracing separates "the model reasoned poorly" from "an external tool/API failed," which need completely different fixes.

COMMON MISTAKES

Logging only that a tool was called, without recording its arguments, result, and timing —

losing the detail needed to actually debug a specific failure.

BEST PRACTICES

Log tool name, arguments, result (or error), and duration for every call; alert on abnormal tool failure or latency rates specifically.

SUMMARY

Tool call tracing isolates the external-system layer of an AI application — critical for correctly diagnosing failures in tool-using systems.

3.6 Retrieval Tracing

DEFINITION

Retrieval tracing is recording exactly what a RAG system (Module 07) retrieved for each query — the query embedding, which chunks were returned, their relevance scores, and what actually made it into the final prompt.

SIMPLE EXPLANATION

This is Module 16's RAG evaluation (offline, on test data) applied continuously to real, live queries — letting you see exactly what your retrieval system found for the actual questions real users asked.

 *Like a librarian keeping a log of exactly which books they pulled for each specific patron's request — if the patron got a bad answer, this log immediately shows whether the wrong books were pulled, or the right books were pulled but misread.*

EVERYDAY EXAMPLE

Tracing a bad RAG answer back to its retrieval step and discovering the correct chunk existed in the knowledge base but scored just below the similarity threshold (Module 07, 3.3) and was never retrieved.

WHY IT MATTERS

Retrieval tracing is what makes Module 16's retrieval-vs-generation failure split (Section 3.8 there) possible on real production traffic, not just curated test data.

COMMON MISTAKES

Only logging the final answer for RAG systems, discarding exactly which chunks were retrieved and their scores.

BEST PRACTICES

Log retrieved chunk IDs, scores, and the final assembled context for every RAG request — this data is invaluable for both debugging and growing your golden dataset (Module 16, 3.2).

SUMMARY

Retrieval tracing shows exactly what a RAG system found for each real query — essential for diagnosing production RAG failures with precision.

3.7 Agent Tracing

DEFINITION

Agent tracing is recording an entire agent's multi-step execution (Module 08) — every think-act-observe cycle, tool call, and decision point, as one connected trace.

SIMPLE EXPLANATION

This combines LLM tracing, tool call tracing, and retrieval tracing together across every step of an agent loop (Module 08, 3.6), since agents can fail at any single step in a chain of many.


```
Agent Trace #892:
Step 1: Think "need to search for pricing"
Step 1: Act    search_web("competitor pricing")      [0.8s]
Step 1: Observe [12 results]
Step 2: Think "extract key numbers"
Step 2: Act    extract_data(...)                    [0.3s]  ✖ FAILED: malformed input
Step 2: Observe [error, retried]
Step 3: Think "retry with cleaned input"
...
```

💡 *Like a flight recorder ("black box") that captures every instrument reading and decision throughout an entire flight — when something goes wrong, investigators can reconstruct the exact sequence of events, not just the final crash.*

EVERYDAY EXAMPLE

Tracing a research agent (Module 14, 3.4) that produced a poor report, and discovering it looped on a failed tool call 3 times before giving up and generating a weaker answer than intended.

WHY IT MATTERS

Agents (Module 08) are the most complex system type in this course to debug — with many steps, any of which can fail — agent tracing is often the only realistic way to understand what actually happened.

COMMON MISTAKES

Only logging an agent's final output, losing the ability to diagnose which specific step in a long chain actually caused a bad result.

BEST PRACTICES

Log every think-act-observe cycle with its own timing and outcome; track step count and loop-detection signals (repeating patterns) as specific health metrics.

SUMMARY

Agent tracing captures an entire multi-step execution as one connected story — the most valuable and most necessary tracing type for genuinely agentic systems.

🚀 3.8 Production Monitoring

DEFINITION

Production monitoring is the ongoing, real-time watching of an AI system's health metrics (from all the tracing types above), with alerts that trigger when something crosses a defined threshold.

SIMPLE EXPLANATION

Tracing gives you the raw data per request; monitoring aggregates that data over time into dashboards and alerts — the difference between having flight-recorder data available and having a cockpit warning light that goes off automatically when something's wrong.

💡 *Like a hospital patient's vital-sign monitors — continuously watching heart rate, oxygen, and blood pressure, and sounding an alarm the moment any reading crosses a dangerous threshold, rather than requiring a nurse to manually check every few hours.*

EVERYDAY EXAMPLE

An alert firing when hallucination-flagged responses (Module 16, 3.7) exceed 5% over a rolling hour, or when p95 latency (Section 3.3) crosses 5 seconds.

WHY IT MATTERS

Without active monitoring and alerts, even perfectly good tracing data sits unused until someone happens to look — monitoring is what makes problems get noticed quickly, automatically.

COMMON MISTAKES

Setting alert thresholds so loose they never fire, or so tight they fire constantly and get ignored ("alert fatigue") — both make monitoring effectively useless.

BEST PRACTICES

Set meaningful, well-calibrated thresholds based on real baseline data; alert on trends (rate of change) as well as absolute values.

SUMMARY

Production monitoring turns raw tracing data into active, real-time awareness — the layer that ensures problems actually get noticed and acted on quickly.

🔧 3.9 Debugging LLM Applications

DEFINITION

Debugging LLM applications is the systematic process of using traces, logs, and monitoring data to diagnose and fix a specific failing case — applying all the previous concepts in this module together.

SIMPLE EXPLANATION

Good debugging follows the trace: start from the symptom (a bad user-facing response), pull its full trace (Section 3.1), and walk backward through each step (retrieval, tool calls, prompt) until you find where things actually went wrong.

💡 *Like following a trail of breadcrumbs backward from where you got lost to the exact fork in the path where you took a wrong turn — rather than staring at your current location and guessing what went wrong somewhere behind you.*

EVERYDAY EXAMPLE

A user reports a wrong answer; you pull the trace, see retrieval (Section 3.6) returned an irrelevant chunk, check retrieval tracing further back, and find the root cause was a chunking boundary issue (Module 07, 3.2) — not a prompt problem at all.

WHY IT MATTERS

Without tracing, debugging an LLM application means guessing which of many possible steps went wrong, often re-running the whole pipeline repeatedly to try to reproduce the issue — tracing turns guessing into direct inspection.

COMMON MISTAKES

Trying to debug from the final output alone, without pulling the full trace, and guessing at fixes instead of diagnosing the actual root cause.

BEST PRACTICES

Always start debugging from the full trace, not the final output alone; once found, add the failing case to your golden dataset (Module 16, 3.2) so it's caught automatically going forward.

SUMMARY

Debugging LLM applications means following the trace backward from symptom to root cause — the practical payoff of everything else in this module.

4. Visual Learning

🔗Observability Stack



📌Evaluation vs. Observability

Knowing Your System Works

- **Evaluation** (Module 16 — controlled test data, before shipping)
- **Observability** (Module 17 — real production traffic, ongoing)

📊Tracing Type Comparison

Tracing Type	Captures	Best For Diagnosing
Prompt tracing	Exact final prompt sent	Template/injection bugs
Tool call tracing	Tool name, args, result, timing	External system failures
Retrieval tracing	Chunks retrieved, scores	RAG retrieval failures
Agent tracing	Full multi-step execution	Complex agent loop failures

5. Real World Applications

Product	How these fundamentals show up
Enterprise chatbots	Cost/latency dashboards with alerts on p95 latency spikes.
RAG-based support tools	Retrieval tracing on every query for ongoing quality monitoring and golden dataset growth.
Coding agents	Agent tracing captures full multi-step executions for post-incident debugging.
High-volume AI APIs	Error tracking trends validation failure rates to catch silent quality drift.

6. Practical Examples

Beginner: Designing a trace schema

Sketch (on paper) what fields a single trace record should contain for a simple chatbot (Module 14, 3.1) — trace ID, timestamp, prompt, response, tokens, latency.

Beginner: Spotting a silent failure

Describe one example of a "silent failure" (something that fails without a user directly noticing) in an AI system you've built or studied in this course.

Intermediate: Setting alert thresholds

For a project from Module 14, propose 3 specific monitoring alerts (metric + threshold) and explain your reasoning for each threshold value.

Advanced: Trace-based debugging exercise

Given a hypothetical trace showing a RAG agent's full execution with one step clearly failed, write out your debugging process: what you'd check first, second, and third.

7. Code Examples

🐍Python — Minimal tracing wrapper around an API call

```
# pip install anthropic
import anthropic
import time
import uuid
import json

client = anthropic.Anthropic(api_key="YOUR_API_KEY")
trace_log = [] # in production: send to a real logging/tracing backend

def traced_call(prompt, step_name="llm_call"):
    trace_id = str(uuid.uuid4())
    start = time.time()

    response = client.messages.create(
        model="claude-sonnet-4-6", max_tokens=300,
        messages=[{"role": "user", "content": prompt}]
    )

    duration = time.time() - start
    trace_log.append({
        "trace_id": trace_id,
        "step": step_name,
        "prompt": prompt, # Module 17, 3.2: prompt tracing
        "response": response.content[0].text,
        "tokens_in": response.usage.input_tokens, # Module 17, 3.3: token tracking
        "tokens_out": response.usage.output_tokens,
        "latency_s": round(duration, 2), # Module 17, 3.3: latency tracking
        "timestamp": time.time(),
    })
    return response.content[0].text, trace_id

result, trace_id = traced_call("Summarize the benefits of solar power.")
print(json.dumps(trace_log[-1], indent=2))
```

📄JavaScript — Simple alert-threshold monitor

```
// A minimal monitoring check — run this periodically against recent trace data
function checkAlerts(recentTraces) {
    const alerts = [];

    const errorRate = recentTraces.filter((t) => t.error).length / recentTraces.length;
    if (errorRate > 0.05) {
        alerts.push(`⚠ Error rate ${errorRate * 100.toFixed(1)}% exceeds 5% threshold`);
    }

    const latencies = recentTraces.map((t) => t.latencyS).sort((a, b) => a - b);
    const p95 = latencies[Math.floor(latencies.length * 0.95)];
    if (p95 > 5) {
        alerts.push(`⚠ p95 latency ${p95}s exceeds 5s threshold`);
    }

    const avgCost = recentTraces.reduce((sum, t) => sum + t.cost, 0) / recentTraces.length;
    if (avgCost > 0.01) {
        alerts.push(`⚠ Average cost $$${avgCost.toFixed(4)} exceeds $0.01 threshold`);
    }

    return alerts;
}

// In production: run this on a schedule and send alerts to Slack/email/pager
const alerts = checkAlerts(recentTraceData);
if (alerts.length > 0) console.log(alerts.join("\n"));
```



Tip

Start with even the simplest possible tracing (an in-memory list, like the Python example) — the habit of tracing every request matters more early on than the sophistication of the tracing backend.

8. Common Mistakes

- **Only logging the final input/output**, discarding intermediate steps needed to diagnose multi-step failures.
- **Only tracking average cost/latency**, missing outliers that may represent a disproportionate share of problems.
- **Only tracking hard crashes**, missing silent failures like validation rejections or successful-but-costly retries.
- **Setting alert thresholds too loose or too tight**, causing either missed problems or alert fatigue.
- **Debugging from the final output alone** instead of pulling the full trace first.
- **Retrofitting observability late**, after a system is already in production and hard to instrument.

9. Best Practices

- **Give every request a unique trace ID** and log every intermediate step against it, from day one.
- **Track distributions** (p50/p95/p99), not just averages, for cost and latency.
- **Track and trend silent failures** alongside hard crashes.
- **Calibrate alert thresholds against real baseline data**, and alert on trends, not just absolute values.
- **Always debug from the full trace**, walking backward from symptom to root cause.
- **Feed discovered production failures back into your golden dataset** (Module 16, 3.2) so they're caught automatically going forward.

10. Hands-on Activities

Easy 5 exercises

1. Design a trace record schema (list the fields) for a simple chatbot.
2. Explain the difference between evaluation (Module 16) and observability (this module) in your own words.
3. List 3 silent failure types that might occur in an AI system, beyond hard crashes.
4. Propose one meaningful alert threshold for a project from Module 14, with reasoning.
5. Sketch the observability stack diagram from Section 4 from memory.

Intermediate 3 exercises

1. Build the Python tracing wrapper from Section 7 and run it against 10 real requests, reviewing the resulting trace log.
2. Add retrieval tracing to a RAG project from Module 07/14, logging retrieved chunks and scores for each query.
3. Build the JS alert-threshold checker from Section 7 and test it against both healthy and unhealthy sample trace data.

Challenge 2 exercises

1. Add full agent tracing (Section 3.7) to an agent project from Module 08/14, capturing every think-act-observe cycle.
2. Take a real (or simulated) production failure and walk through the full trace-based debugging process from Section 3.9, documenting each step of your investigation.

11. Mini Project

Goal

Add a complete observability layer to a project from Module 14 — tracing, metrics tracking, and basic alerting, end to end.

Requirements

- An Anthropic API key
- Python or JavaScript
- An existing project from Module 14 (or a new simple one)

Step-by-Step Instructions

1. Add trace ID generation and full request/response logging (Section 7's pattern) to your project.
2. Track tokens, cost, and latency per request, storing them for later analysis.
3. If your project uses tools or retrieval, add tool call tracing and/or retrieval tracing specifically.
4. Build a simple alert checker (Section 7's JS pattern) with 2-3 meaningful thresholds.
5. Run your project 15-20 times with varied inputs, then review the aggregated trace data: any surprises, outliers, or patterns?

Expected Output

```
Traces collected: 18
Avg latency: 1.6s | p95 latency: 3.9s
Avg cost/request: $0.0021
Errors: 1 (validation rejection, request #14)

Alerts triggered: none
Notable pattern: request #7 and #14 both had unusually
long input — worth checking if this correlates with errors.
```

Possible Improvements

- Connect to a real observability platform (e.g., an LLM-specific tracing tool) instead of in-memory logging.
- Set up automated alerting to a real channel (Slack, email) instead of just console output.
- Use discovered outliers/failures to grow your Module 16 golden dataset.

12. Review Questions

Multiple Choice (10)

1. LLM tracing primarily captures:

- A) Only the final output B) The complete path a request takes through a system C) Only the model's training data D) Nothing useful

2. Prompt tracing is especially important because:

- A) Prompts never change B) Dynamically assembled prompts can differ from what code alone suggests C) It replaces all other tracing D) It reduces token cost

3. Why track latency distributions (p95, p99) instead of just averages?

- A) Averages are always sufficient B) Averages can hide meaningful outliers C) Distributions are irrelevant D) It's required by every API

4. A "silent failure" is best described as:

- A) A hard crash B) A failure that doesn't immediately surface to the user but still matters C) A successful request D) A type of model

5. Tool call tracing helps distinguish:

- A) Nothing useful B) Model reasoning failures from external tool/API failures C) Only cost issues D) Only latency issues

6. Retrieval tracing is most useful for diagnosing:

- A) Agent loop bugs only B) RAG-specific failures (wrong or missing chunks) C) Model training issues D) UI bugs

7. Agent tracing typically combines:

- A) Nothing else B) LLM, tool call, and retrieval tracing across every step C) Only cost tracking D) Only the final answer

8. Production monitoring's main role is to:

- A) Replace tracing entirely B) Aggregate data and alert when thresholds are crossed C) Train new models D) Increase temperature

9. Effective LLM application debugging should start from:

- A) Guessing B) The full trace, walking backward from symptom to root cause C) The final output only D) Retraining the model

10. Alert thresholds that are too tight commonly cause:

- A) Perfect monitoring B) Alert fatigue, where alerts get ignored C) Lower cost D) No effect at all

Identification (5)

- _____ records the complete path a single request takes through an AI system.
- _____ records the exact final prompt sent to the model for a given request.
- _____ records every tool invocation's name, arguments, result, and timing.
- _____ aggregates traced data into dashboards and real-time alerts.
- _____ is the systematic process of using traces to diagnose and fix a failing case.

True or False (5)

- Evaluation (Module 16) and observability (this module) serve the same exact purpose.
- Tracing only the final input/output is sufficient for debugging multi-step failures.
- Retrieval tracing helps determine whether a RAG failure was a retrieval or generation problem.
- Alert thresholds should be calibrated against real baseline data.
- Discovered production failures should be fed back into a golden dataset for future evaluation.

Situational (5)

- A user reports a wrong RAG answer, and you need to find out why. What's

your first debugging step?

2. Your average latency looks fine, but some users report slow responses. What tracking approach would reveal this?

3. Your agent occasionally produces poor results, and you suspect a specific tool is unreliable. What tracing type would confirm this?

4. Your alerts fire constantly and the team has started ignoring them. What's the likely problem?

5. You found a real production failure while debugging. What should you do with it afterward?

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-LLM tracing, 2-Prompt tracing, 3-Tool call tracing, 4-Production monitoring, 5-Debugging LLM applications

True or False: 1-False, 2-False, 3-True, 4-True, 5-True

Situational (sample reasoning):

1. Pull the full trace for that request and walk through each step (retrieval, prompt, generation).
2. Tracking latency distributions (p95/p99), not just the average.
3. Tool call tracing — showing that specific tool's failure/timeout rate.
4. Alert thresholds are miscalibrated (too tight), causing alert fatigue.
5. Add it to the golden dataset (Module 16, 3.2) so it's caught automatically in future evaluations.

13. Cheat Sheet

 Core Tracing

Every request gets a trace ID. Log the actual final prompt, not just the template.

 Monitoring

Track distributions, not just averages. Calibrate thresholds against real baselines.

 Specialized Tracing

Tool calls, retrieval, and full agent loops each need their own dedicated tracing.

 Debugging

Start from the full trace, walk backward to root cause. Feed findings back into evaluation.

14. Key Takeaways

- Observability watches real production traffic; evaluation (Module 16) tests controlled data — both are needed for full confidence.
- LLM tracing captures every step of a request's journey; prompt tracing specifically captures the actual, final assembled prompt.
- Track token, cost, and latency as distributions in production, not just pre-launch estimates or simple averages.
- Error tracking must include silent failures, not just hard crashes.
- Tool call, retrieval, and agent tracing each isolate a specific layer for precise failure diagnosis.
- Production monitoring turns raw trace data into active, real-time alerts — calibrated thresholds matter enormously.
- Effective debugging always starts from the full trace, walking backward from symptom to root cause.

15. Additional Resources

Official Documentation

- Anthropic — usage and monitoring guidance (docs.claude.com)

Tools

- LangSmith, Braintrust, Helicone, and similar LLM-specific observability/tracing platforms.
- General-purpose observability tools (e.g., OpenTelemetry) adapted for LLM application tracing.

Related Modules

- Module 16 — LLM Evaluation & Testing (the offline counterpart to this module's production observability)
- Module 09 — Prompt Optimization (the cost/latency fundamentals this module tracks continuously)

Communities & Further Learning

- LLM observability platform blogs and case studies for production-scale monitoring patterns.